

SURE-EVAL 用户使用手册

Systematic Unified Robust Evaluation Framework for Audio Processing

版本: v2.0

日期: 2026-06-16

 **版本说明**：本手册已随 [docs/agents/](#) 目录重构同步更新。主流程 Agent 文档现位于 [docs/agents/main_flow_agent/](#)，模型接入 Agent 文档现位于 [docs/agents/model_tool_agent/](#)。

目录

1. SURE-EVAL 是什么
2. 环境准备与安装
3. 数据集准备
4. 模型接入：以 Qwen3-ASR 为参考示例
5. 评估执行方式
6. Agent Flow 详解
7. 推理协议系统
8. 结果解读与报告
9. 常见问题与故障排除

1. SURE-EVAL 是什么

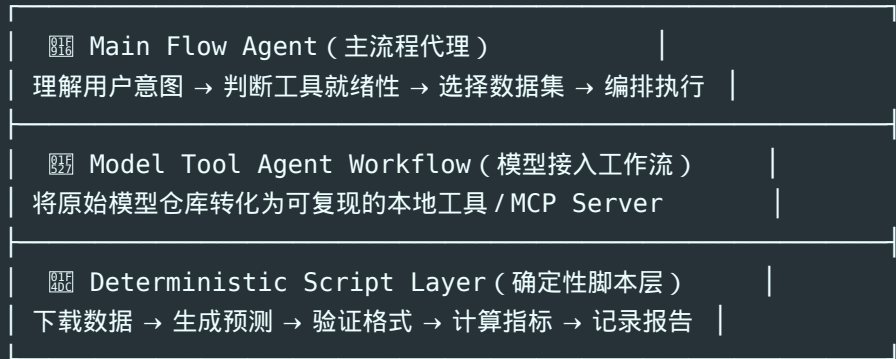
1.1 核心理念

SURE-EVAL (**S**ystematic **U**nified **R**obust **E**valuation) 是一个面向音频处理模型的自动化评测框架。它的设计哲学非常简单：

Agent 决定范围，脚本强制执行。

换句话说，复杂的不确定性决策（选什么模型、用什么数据集、怎么跑）交给 Agent 来判断；而具体的执行步骤（下载数据、生成预测、计算指标）则交给确定性的脚本，确保每次执行结果可复现、可审计。

1.2 三层架构



三层的关系： - **脚本层**是基础——无论有没有 Agent，你都可以直接调用这些脚本完成评估。 - **工具接入层**是桥梁——把 GitHub 上的模型仓库变成 SURE-EVAL 能调用的本地工具。 - **Agent 层**是编排器——它判断该做什么、不该做什么，然后把工作分发给脚本层。

两个 Agent Workflow 的选择方式：

用户目标

- └─ 已经有可调用的模型目录
 - └─ 使用 `main_flow_agent` : 选择数据集、生成执行入口、运行评估、产出报告
- └─ 模型还没有完成接入，或环境 / `wrapper` / `contract` 未通过
 - └─ 使用 `model_tool_agent` : 接入模型、修复环境、生成 `wrapper`、完成验证
- └─ 不确定是否就绪
 - └─ 先让 `main_flow_agent` 做 `readiness gate`，不就绪时转交 `model_tool_agent`

详细流程图见 [docs/agents/workflow_gallery.md](#)。

1.3 支持的评测任务

ASR	自动语音识别	AISHELL-1, LibriSpeech, KeSpeech
S2TT	语音到文本翻译	CoVoST2 (EN→ZH, ZH→EN)
SD	说话人分割	
SER	语音情感识别	IEMOCAP
Speech Enhancement	语音增强	
Music IR	音乐信息检索	

1.4 Qwen3-ASR : 参考示例

本手册以 **Qwen3-ASR-1.7B** 作为贯穿始终的参考示例。它已经在 SURE-EVAL 中完成接入，拥有完整的 `config.yaml`、`model.py`、`server.py` 和评估记录，是最适合用来理解整个流程的模型。

2. 环境准备与安装

2.1 系统要求

Python	3.10+ (主环境) ; 部分模型需要 3.11)
GPU	推荐但非必需 (Qwen3-ASR 可用 CPU , 但速度较慢)
磁盘空间	至少 20GB (模型权重 + 数据集)
网络	可访问 ModelScope (HuggingFace 在国内集群通常被屏蔽)

2.2 安装主环境

SURE-EVAL 使用 `uv` 作为包管理工具 , 比 `pip` 更快、更可靠。

```
# 1. 克隆仓库
git clone <your-repo-url> sure-eval
cd sure-eval

# 2. 创建虚拟环境 ( 主环境使用 Python 3.12 )
uv venv --python 3.12
source .venv/bin/activate

# 3. 安装依赖
uv pip install -e .
```

2.3 验证安装

```
# 查看已接入的模型
python -m sure_eval.models.registry

# 预期输出示例 :
# =====
# SURE-EVAL Model Registry
# =====
# ✓ asr_qwen3           [ASR      ] Automatic Speech Recognition using Qwen3-
ASR-1.7B...
# ✓ asr_whisper         [ASR      ] OpenAI Whisper base model...
# ...
```

2.4 模型本地环境

每个模型有自己的独立虚拟环境（位于 `src/sure_eval/models/<model>/.venv/`），与主环境隔离。以 Qwen3-ASR 为例：

```
cd src/sure_eval/models/asr_qwen3

# 查看 setup.sh 了解环境创建步骤
cat setup.sh

# 通常包括：
# uv venv --python 3.11
# source .venv/bin/activate
# uv pip install -e .
```

3. 数据集准备

3.1 SURE Benchmark 数据集

SURE-EVAL 的核心评测数据来源是 **SURE Benchmark**，托管在 ModelScope 上。包含：

- **SURE_Test_csv**：标注文件（CSV 格式，约 50MB）
- **SURE_Test_Suites**：音频文件（约 11GB，含多个子集）

一键下载

```
cd /path/to/sure-eval

# 下载全部数据
python scripts/download_sure_data.py

# 仅下载标注文件
python scripts/download_sure_data.py --csv

# 仅下载音频文件
python scripts/download_sure_data.py --suites

# 验证数据完整性
python scripts/download_sure_data.py --csv --suites
# 或分别下载后检查 data/datasets/sure_benchmark/ 目录是否存在
```

手动下载（如果脚本不可用）

```
pip install modelscope

# 下载标注
modelscope download \
  --dataset SUREBenchmark/SURE_Test_csv \
  --local_dir ./data/datasets/sure_benchmark/SURE_Test_csv

# 下载音频
modelscope download \
  --dataset SUREBenchmark/SURE_Test_Suites \
  --local_dir ./data/datasets/sure_benchmark/SURE_Test_Suites

# 解压音频
cd ./data/datasets/sure_benchmark/SURE_Test_Suites
for f in *.tar.gz; do
  mkdir -p "${f%.tar.gz}"
  tar -xzf "$f" -C "${f%.tar.gz}"
done
```

数据集统计

aishell-1_test	7,175	1.08 GB	ASR	中文
librispeech-test-clean	2,619	0.58 GB	ASR	英文
librispeech-test-other	2,939	0.57 GB	ASR	英文
CoVoST2_S2TT_en2zh_test	15,530	2.65 GB	S2TT	EN→ZH
IEMOCAP_test	-	215 MB	SER	英文

3.2 OREF 本地数据集适配

除了 SURE Benchmark，SURE-EVAL 也支持使用本地已有的 OREF 格式数据集。本地数据集的目录结构如下：

```

data/datasets/
├── sure_benchmark/          # SURE 官方数据集
│   ├── SURE_Test_csv/
│   └── SURE_Test_Suites/
├── librispeech_test_clean/  # 本地 OREF 数据集示例
│   ├── audio/              # 音频文件
│   └── sample.jsonl        # 样本描述文件
└── aishell1_test/          # 另一个本地 OREF 数据集
    ├── audio/
    └── sample.jsonl

```

OREF JSONL 格式

每个 `sample.jsonl` 文件的每一行是一个 JSON 对象：

```

{"key": "audio_file_id", "path": "audio/audio_file.wav", "text": "对应的文本标注",
"task": "ASR"}

```

字段说明： - **key**：样本唯一标识，用于匹配预测结果 - **path**：音频文件相对路径（相对于数据集目录） - **text**：参考答案（ground truth） - **task**：任务类型（ASR、S2TT 等）

将 OREF 数据接入 SURE-EVAL

如果你已有 OREF 格式的本地数据，只需确保目录结构符合上述格式，SURE-EVAL 的 `DatasetManager` 会自动识别。你也可以通过配置文件显式声明：

```

# 在 sure-eval 配置中声明本地数据集
datasets:
  definitions:
    my_local_asr:
      task: ASR
      language: zh
      source: oref_local
      path: data/datasets/my_local_asr

```

4. 模型接入：以 Qwen3-ASR 为参考示例

4.1 模型目录结构

一个完整接入的模型目录结构如下（以 `asr_qwen3` 为例）：

```

src/sure_eval/models/asr_qwen3/
├─ README.md          # 模型说明文档
├─ config.yaml        # MCP 配置（核心文件）
├─ model.py           # 模型 wrapper
├─ server.py          # MCP server（推理入口）
├─ model.spec.yaml    # 模型规格说明
├─ pyproject.toml     # 依赖定义
├─ setup.sh           # 环境初始化脚本
├─ __init__.py        # 包导出
├─
├─ .venv/              # 模型独立虚拟环境
├─ .runtime/           # 运行时缓存（权重等）
├─   └─ modelscope_cache/
├─
├─ checkpoints/        # 本地权重（如有）
├─ artifacts/          # 生成的构建产物
├─   └─ build_plan.json
├─   └─ weights_manifest.json
├─   └─ verdict.json
├─
├─ eval_runs/          # 评估运行记录
├─   └─ main_agent_asr_qwen3_001/
├─     └─ predictions/      # 预测结果
├─     └─ assessment_report.json
├─     └─ run_evaluation.sh # 可复现的执行脚本
├─
└─ results/            # 历史测试结果

```

4.2 config.yaml 详解

`config.yaml` 是模型接入的核心配置文件，决定了 SURE-EVAL 如何调用你的模型。


```

# ASR Qwen3 Model Configuration

name: asr_qwen3
task: ASR
description: "Automatic Speech Recognition using Qwen3-ASR-1.7B"
version: "1.0.0"

# 模型信息
model:
  id: "Qwen/Qwen3-ASR-1.7B"      # ModelScope / HuggingFace 模型 ID
  size: "1.7B"
  languages: ["zh", "en", "ja", "ko", "auto"]
  license: "Apache 2.0"

# MCP Server 配置
server:
  command: [".venv/bin/python", "server.py"] # 启动命令
  working_dir: "."                          # 工作目录
  env:                                       # 环境变量
    MODEL_PATH: "Qwen/Qwen3-ASR-1.7B"
    MODELSCOPE_CACHE: "/path/to/modelscope_cache"
    DEVICE: "auto"
  timeout: 300
  startup_timeout_sec: 300

# 工具定义
tools:
  - name: "asr_transcribe"
    description: "Transcribe speech to text"
    input_schema:
      type: object
      properties:
        audio_path:
          type: string
          description: "Path to audio file"
        language:
          type: string
          description: "Language (Chinese, English, auto)"
          default: "auto"
      required: ["audio_path"]

# 协议声明（新增，可选但推荐）
protocols:
  strict_core:
    enabled: true
  param_map:

```

```
precision:
  model_param: dtype
  mapping:
    float16: "torch.float16"
    float32: "torch.float32"
    bfloat16: "torch.bfloat16"
max_batch_size:
  model_param: batch_size
  mapping:
    "1": 1
    "4": 1
    "8": 1
    "16": 1

# 资源需求
resources:
  memory_gb: 8
  gpu: true
  cpu_cores: 4
  storage_gb: 4
```

4.3 model.py : 模型 Wrapper

`model.py` 是模型的核心封装，必须实现一个包含 `transcribe()` 方法的类。以 Qwen3-ASR 为例：

```

class ASRQwen3Model:
    def __init__(self, model_path: str, device: str = "auto"):
        # 加载模型和处理器
        self.model = AutoModel.from_pretrained(model_path, ...)
        self.processor = AutoProcessor.from_pretrained(model_path, ...)

    def transcribe(self, audio_path: str, language: str | None = None):
        """核心推理方法。

        Args:
            audio_path: 音频文件路径
            language: 语言代码（如 "Chinese", "English"）, None 表示自动检测

        Returns:
            包含 text, language, timestamps 的结果对象
        """
        # 加载音频
        # 调用模型推理
        # 返回结果
        return Result(text="...", language="zh", timestamps=[...])

```

关键要求： - 类名可以自定义，但 `server.py` 需要能正确找到它 - `transcribe()` 必须接受 `audio_path` 参数 - 返回对象必须至少有 `.text` 属性

4.4 server.py : MCP Server

`server.py` 是一个基于 JSON-RPC 的 MCP (Model Context Protocol) 服务器，负责：1. 接收外部调用请求 2. 加载模型（懒加载） 3. 执行推理 4. 返回结果

它通过标准输入/输出与调用方通信，因此可以被 SURE-EVAL 的脚本直接调用。

通信协议： - 输入：JSON-RPC 2.0 请求（每行一个 JSON 对象） - 输出：JSON-RPC 2.0 响应（每行一个 JSON 对象）

示例请求：

```

{"jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": {"name":
"asr_transcribe", "arguments": {"audio_path": "/path/to/audio.wav", "language":
"auto"}}}

```

4.5 从零接入一个新模型

如果你的模型还没有接入 SURE-EVAL，有两种方式：

方式 A : Agent 自动接入 (推荐)

使用 Model Tool Agent 自动完成接入 :

- 1. 准备 `MODEL_INPUT` , 描述你的模型信息
- 2. 发送给 Agent , 让它执行完整的工作流
- 3. Agent 会自动生成 `model.py` 、 `server.py` 、 `config.yaml` 等文件

具体输入格式参考 `docs/agents/model_tool_agent/README.md` (./agents/model_tool_agent/README.md) 中的 `## MODEL_INPUT` 部分。

方式 B : 手动接入

如果你更熟悉代码 , 可以手动创建以下文件 :

- 1. 创建目录 : `src/sure_eval/models/my_model/`
- 2. 编写 `model.py` : 实现模型 wrapper 类
- 3. 编写 `server.py` : 实现 MCP server
- 4. 编写 `config.yaml` : 声明模型信息和工具
- 5. 编写 `pyproject.toml` : 声明依赖
- 6. 编写 `setup.sh` : 环境初始化脚本
- 7. 测试 : 运行 `python -m sure_eval.models.registry` 确认模型出现

5. 评估执行方式

SURE-EVAL 提供两种评估执行方式 :

手动脚本执行	开发调试、快速验证、CI/CD	确定性强、可复现、无交互
Agent Flow 执行	正式评估、多数据集批量跑、报告生成	智能编排、交互确认、结构化输出

5.1 手动脚本执行 (确定性流程)

这是最直接的方式 , 适合理解底层流程和快速验证。

Step 1 : 准备数据集

```
python scripts/prepare_sure_dataset.py --dataset aishell1
```

这会下载并转换数据集，生成 `sample.jsonl` 文件。

Step 2 : 生成预测模板

```
python scripts/materialize_predictions_template.py \  
  --dataset aishell1 \  
  --output-dir /tmp/predictions
```

这会生成一个空的预测文件模板（`aishell1.txt`），格式为 `key<TAB>prediction`。

Step 3 : 生成预测（通过模型 Server）

```
python scripts/generate_predictions_via_server.py \  
  --model-dir src/sure_eval/models/asr_qwen3 \  
  --dataset aishell1 \  
  --run-dir /tmp/eval_run \  
  --tool-name asr_transcribe \  
  --language auto
```

关键参数说明： - `--model-dir`：模型目录路径 - `--dataset`：数据集名称 - `--run-dir`：运行输出目录 - `--tool-name`：要调用的工具名称（在 `config.yaml` 的 `tools` 中定义） - `--language`：语言参数（传递给模型） - `--resume`：断点续跑（已完成的样本会跳过） - `--max-samples`：限制样本数（快速测试） - `--protocol`：推理协议 ID（默认 `strict_core`，详见第 7 章）

Step 4 : 验证预测文件

```
python scripts/validate_prediction_files.py \  
  --dataset aishell1 \  
  --pred-dir /tmp/eval_run/predictions \  
  --require-nonempty
```

Step 5 : 计算指标

```
python scripts/evaluate_predictions.py \  
  --dataset aishell1 \  
  --pred-dir /tmp/eval_run/predictions \  
  --tool-name asr_qwen3 \  
  --record \  
  --output /tmp/eval_result.json
```

这会输出： - **WER**（词错误率）或 **CER**（字错误率） - **RPS**（Relative Performance Score，相对性能分数） - 与 SOTA 基线的对比

完整流程示例

```
# 1. 准备数据
python scripts/prepare_sure_dataset.py --dataset aishell1

# 2. 创建运行目录
mkdir -p /tmp/eval_run/predictions

# 3. 生成预测 (Qwen3-ASR on AISHELL-1)
python scripts/generate_predictions_via_server.py \
    --model-dir src/sure_eval/models/asr_qwen3 \
    --dataset aishell1 \
    --run-dir /tmp/eval_run \
    --tool-name asr_transcribe \
    --language auto \
    --resume

# 4. 验证
python scripts/validate_prediction_files.py \
    --dataset aishell1 \
    --pred-dir /tmp/eval_run/predictions \
    --require-nonempty

# 5. 评估
python scripts/evaluate_predictions.py \
    --dataset aishell1 \
    --pred-dir /tmp/eval_run/predictions \
    --tool-name asr_qwen3 \
    --record \
    --output /tmp/eval_result.json

# 6. 刷新报告快照
python scripts/refresh_report_snapshot.py \
    --markdown reports/asr_qwen3.md \
    --json reports/asr_qwen3_summary.json
```

5.2 Agent Flow 执行（交互式流程）

Agent Flow 是 SURE-EVAL 的高级用法，适合正式评估和批量跑多数据集。

基本思路

1. 你向 Agent 描述目标（如“评估 Qwen3-ASR 在 AISHELL-1 和 LibriSpeech 上的表现”）
2. Agent 按照预定义的状态机逐步执行

3. 每个关键节点，Agent 会生成结构化文件并可能请求你确认
4. 最终生成完整的评估报告

启动 Agent Flow

准备以下输入（称为 `MAIN_FLOW_INPUT`）：

```
MAIN_FLOW_INPUT:
  user_goal: evaluate_existing_model

  target:
    model_name: asr_qwen3
    model_dir: src/sure_eval/models/asr_qwen3
    tool_workflow_ready: true
    integration_state: onboarded

  constraints:
    allow_tool_workflow: true
    allowed_tasks: [ASR]
    allowed_datasets: null      # null 表示不限制
    blocked_datasets: []
    dry_run: false

  evidence:
    readme_path: src/sure_eval/models/asr_qwen3/README.md
    config_path: src/sure_eval/models/asr_qwen3/config.yaml
    artifacts_dir: src/sure_eval/models/asr_qwen3/artifacts
    model_spec_path: src/sure_eval/models/asr_qwen3/model.spec.yaml
    prior_results: []

  runtime_context:
    available_scripts:
      - scripts/prepare_sure_dataset.py
      - scripts/materialize_predictions_template.py
      - scripts/generate_predictions_via_server.py
      - scripts/validate_prediction_files.py
      - scripts/evaluate_predictions.py
      - scripts/check_execution_surface_compliance.py
      - scripts/refresh_report_snapshot.py
    output_dir: src/sure_eval/models/asr_qwen3/eval_runs/
    main_agent_asr_qwen3_001
```

然后将上述输入发送给 Agent（具体方式取决于你使用的 AI 工具）。

6. Agent Flow 详解

6.1 Workflow 选择图

```
用户请求
↓
当前目标是什么？
├─ 评估已接入模型
│   └─ main_flow_agent
├─ 接入新模型 / 修复已有模型工具
│   └─ model_tool_agent
└─ 不确定
    └─ main_flow_agent readiness gate
        ├── tool ready → main_flow_agent
        └── not ready / broken → model_tool_agent
```

6.2 Main Flow Agent 状态机

Main Flow Agent 用于“模型已经可调用，需要正式评估”的场景。它的执行遵循一个严格的状态机，不允许跳过任何关键步骤：

```
INTAKE (接收用户输入)
↓
TASK_CLASSIFICATION_UNIT (任务分类)
↓
TOOL_READINESS_AND_ROUTING_UNIT (工具就绪性判断)
↓
PLAN_UNIT (制定执行计划)
↓
DATASET_SCOPE_UNIT (选择数据集范围)
↓
SCRIPT_ROUTING_UNIT (编排脚本执行顺序)
↓
EXECUTION_SURFACE_UNIT (生成执行入口)
↓
EXECUTION_READINESS_UNIT (执行前验证)
↓
SMOKE_TEST_UNIT (冒烟测试)
↓
EXECUTE_SCRIPTS / WAIT_FOR_TOOL_WORKFLOW (正式执行)
↓
ASSESSMENT_UNIT (结果评估)
↓
RUN_REPORT_UNIT (生成报告)
↓
DONE
```

6.3 Model Tool Agent 状态机

Model Tool Agent 用于“模型还不能被 SURE-EVAL 稳定调用”的场景。它负责把原始模型仓库或半成品模型目录整理成可验证、可复现、可被 main flow 调用的工具。

```
DISCOVER (收集模型、仓库、环境、权重证据)
↓
CONTEXT_SELECTION_UNIT (按任务、环境、失败类型选择文档)
↓
CLASSIFY (确认任务类型和接入目标)
↓
PLAN (制定接入 / 修复计划)
↓
VALIDATE_SPEC (验证 model.spec.yaml)
↓
BUILD_ENV (创建 uv / pip / conda / docker 环境)
↓
FETCH_WEIGHTS (准备模型权重和缓存)
↓
VALIDATE_ENV_COMPAT (验证运行环境兼容性)
↓
VALIDATE_IMPORT (验证依赖导入)
↓
VALIDATE_LOAD (验证模型加载)
↓
VALIDATE_INFER (验证单样本推理)
↓
VALIDATE_CONTRACT (验证输入输出契约)
↓
GENERATE_WRAPPER (生成 model.py / server.py / validate.py)
↓
SAVE_ARTIFACTS (保存日志、验证记录和交接文件)
↓
DOCKER_BUILD_VALIDATE (如需要, 构建并验证镜像)
↓
TOOL_READY (交回 main_flow_agent)
```

CONTEXT_SELECTION_UNIT 只决定读取哪些任务 playbook、环境 playbook、fixture 索引和 bad-case 记忆；它不会改变上述状态机。

6.4 各 UNIT 详解

TASK_CLASSIFICATION_UNIT

作用：判断当前请求属于哪类任务。

<code>evaluate_existing_model</code>	评估已接入的模型
<code>onboarding_then_evaluate</code>	先接入模型，再评估
<code>repair_broken_model</code>	修复已接入但损坏的模型
<code>audit_results</code>	审计已有评估结果

输出文件： `task_classification.json`

用户交互点： 通常无需交互，Agent 自动判断。

TOOL_READINESS_AND_ROUTING_UNIT

作用： 判断模型当前是否可以直接使用，还是需要修复/接入。

判断逻辑：

```

检查 config.yaml 中的 server 配置
↓
是否有声明 server 路径？
├── 是 → 检查 server.py 和 model.py 是否存在
│   ├── 都存在 → server_ready
│   └── 缺失 → tool_broken_needs_repair
└── 否 → not_tool_ready (需要接入)

```

输出文件： `tool_readiness_routing.json`

关键字段： - `tool_readiness_state` : `server_ready` / `server_declared_but_unverified` / `tool_broken_needs_repair` / `not_tool_ready` - `preferred_execution_path` : `direct_server_use` / `server_smoke_test_first` / `handoff_to_tool_workflow` - `handoff_to_tool_agent` : true/false (是否需要转交工具接入 Agent)

用户交互点： - 如果判断为 `tool_broken_needs_repair`，Agent 会停止评估并建议你修复模型 - 如果判断为 `not_tool_ready`，Agent 会建议你先走 Model Tool Agent 流程

PLAN_UNIT

作用：形成本轮评估的总体计划。

输出文件： `main_agent_plan.json`

内容示例：

```
{
  "goal": "Evaluate asr_qwen3 on AISHELL-1 and LibriSpeech test-clean",
  "steps": [
    "Prepare datasets (aishell1, librispeech_test_clean)",
    "Run smoke test with 10 samples",
    "Generate predictions via MCP server",
    "Validate prediction files",
    "Compute WER/CER and RPS",
    "Generate run report"
  ],
  "estimated_duration": "2-3 hours",
  "stop_conditions": ["all datasets evaluated", "smoke test fails"]
}
```

用户交互点：Agent 会展示计划，你可以确认或调整。

DATASET_SCOPE_UNIT

作用：根据模型能力和用户约束，选择要评估的数据集。

决策逻辑：1. 读取 `config.yaml` 中的 `model.languages` 和 `task` 2. 过滤匹配的数据集 3. 应用用户的 `allowed_datasets` 和 `blocked_datasets` 约束 4. 排除模型不支持的语言/任务

输出文件： `dataset_decision.json`

内容示例：

```
{
  "selected_datasets": ["aishell1", "librispeech_test_clean"],
  "skipped_datasets": [
    {
      "dataset": "CoVoST2_S2TT_en2zh_test",
      "reason": "Model task is ASR, not S2TT"
    },
    {
      "dataset": "IEMOCAP_SER_test",
      "reason": "Model task is ASR, not SER"
    }
  ]
}
```

用户交互点： - Agent 会列出选中的数据集合和跳过的数据集及原因 - 你可以要求添加或删除数据集

SCRIPT_ROUTING_UNIT

作用： 将评估计划转化为具体的脚本调用序列。

输出文件： `script_routing.json`

内容示例：

```
{
  "steps": [
    {
      "step": 1,
      "script": "scripts/prepare_sure_dataset.py",
      "args": ["--dataset", "aishell1"],
      "output": "data/datasets/sure_benchmark/jsonl/aishell1.jsonl"
    },
    {
      "step": 2,
      "script": "scripts/generate_predictions_via_server.py",
      "args": ["--model-dir", "src/sure_eval/models/asr_qwen3", ...],
      "depends_on": [1]
    }
  ]
}
```

用户交互点： 通常无需交互，Agent 自动编排。

EXECUTION_SURFACE_UNIT

作用：生成最终的执行入口（通常是一个 shell 脚本）。

输出文件： - `execution_surface.json`：执行面描述 - `run_evaluation.sh`：可直接运行的 shell 脚本

run_evaluation.sh 示例：

```
#!/bin/bash
set -euo pipefail

REPO_ROOT=$(cd "$(dirname "$0")/../../.." && pwd)
MODEL_DIR="$REPO_ROOT/src/sure_eval/models/asr_qwen3"
RUN_DIR="$REPO_ROOT/src/sure_eval/models/asr_qwen3/eval_runs/
main_agent_asr_qwen3_001"

# 准备数据
cd "$REPO_ROOT"
python scripts/prepare_sure_dataset.py --dataset aishell1

# 生成预测
python scripts/generate_predictions_via_server.py \
    --model-dir "$MODEL_DIR" \
    --dataset aishell1 \
    --run-dir "$RUN_DIR" \
    --tool-name asr_transcribe \
    --language auto \
    --resume

# 验证
python scripts/validate_prediction_files.py \
    --dataset aishell1 \
    --pred-dir "$RUN_DIR/predictions" \
    --require-nonempty

# 评估
python scripts/evaluate_predictions.py \
    --dataset aishell1 \
    --pred-dir "$RUN_DIR/predictions" \
    --tool-name asr_qwen3 \
    --record \
    --output "$RUN_DIR/eval_result.json"
```

用户交互点： - Agent 会展示生成的 shell 脚本 - 你可以审查并确认后再执行

EXECUTION_READINESS_UNIT

作用：在正式执行前做预检，确保不会跑到一半失败。

检查项：1. shell 脚本语法是否正确 2. 模型 server 能否启动 3. 数据集是否存在 4. 输出目录是否可写 5. **冒烟测试**：用少量样本验证模型能产出非空预测

输出文件： `execution_readiness_report.json`

关键字段： - `execution_ready` : true/false - `smoke_test_passed` : true/false - `blocking_issues` : 阻塞问题列表

用户交互点： - **这是阻塞性检查**。如果冒烟测试失败，Agent 会停止并报告问题，不会进入正式执行。 - 如果 resume 检查发现已有部分预测文件，Agent 会确认 `--resume` 是否正确衔接。

SMOKE_TEST_UNIT

作用：bounded 冒烟测试，验证模型在真实数据上能产出非空预测。

流程：1. 从数据集中取少量样本（如 10 条）2. 调用模型生成预测 3. 检查预测是否非空 4. 如果通过，才允许进入全量执行

输出文件： `smoke_test_report.json`

用户交互点： - 如果冒烟测试失败，Agent 会停止并让你检查模型 - 你可以要求调整样本数或跳过（不推荐）

ASSESSMENT_UNIT

作用：评估执行结果，检测异常。

异常检测规则： - WER / CER > 50% → 异常，暂停确认 - Accuracy < 20% → 异常，暂停确认 - 预测文件为空 → 异常，暂停确认 - 样本缺失 → 异常，暂停确认

输出文件： `assessment_report.json`

用户交互点： - **这是最重要的交互点之一** - 如果检测到异常，Agent 会暂停并询问你是否继续 - 你需要确认"结果看起来合理"后，Agent 才会继续

RUN_REPORT_UNIT

作用：生成最终的运行报告。

流程：1. 汇总所有结构化输出 2. 生成报告预览 3. **请求用户确认** (y/n) 4. 确认后才持久化到文件

输出文件： - `main_agent_run_report.json` - `model_eval_manifest.json` (单文件索引，包含本轮所有证据)

用户交互点： - Agent 会展示报告预览 - 你需要输入 `y` 确认后才保存 - 如果拒绝，报告会标记为 `cancelled`

6.5 交互点总览

TASK_CLASSIFICATION	🔧 自动	无
TOOL_READINESS_AND_ROUTING	⚠ 可能	模型未就绪时停止
PLAN_UNIT	⚠ 可能	确认或调整计划
DATASET_SCOPE	⚠ 可能	确认数据集范围
SCRIPT_ROUTING	🔧 自动	无
EXECUTION_SURFACE	⚠ 可能	审查 shell 脚本
EXECUTION_READINESS	🚫 阻塞	冒烟测试失败时停止
SMOKE_TEST	🚫 阻塞	测试结果异常时停止
ASSESSMENT	🚫 阻塞	指标异常时请求确认
RUN_REPORT	🔧 必须	预览后确认才保存

7. 推理协议系统

7.1 为什么需要推理协议

不同的评测报告可能会使用不同的推理参数（如温度、beam search、批大小），导致结果不可比。推理协议系统旨在**统一这些参数**，让评测结果具有可比性。

7.2 两个协议

Protocol A	<code>strict_core</code>	最严格的纯模型能力测试	默认，论文 / 基准测试
Protocol B	<code>standard_system</code>	允许标准系统组件	系统部署评估

默认使用 `strict_core`。

7.3 Protocol A (`strict_core`) 的四个统一约束

1. 搜索强度统一：所有模型使用默认解码策略
2. 上下文使用统一：不允许注入外部上下文
3. 外部信息统一：禁止调用外部语言模型、检索、热词等
4. 计算预算统一：精度（float16/float32）、最大批大小统一

7.4 模型自映射机制

不同模型的 API 参数名不同。SURE-EVAL 采用"协议标准参数名 + 模型自声明映射"的机制：

```
# config.yaml 中的协议声明示例
protocols:
  strict_core:
    enabled: true
    param_map:
      precision:
        model_param: dtype
        mapping:
          float16: "torch.float16"
          float32: "torch.float32"
      max_batch_size:
        model_param: batch_size
      mapping:
        "1": 1
        "4": 1
```

对于不支持的参数，显式声明为 `null`：

```
search_strength:
  model_param: null
  note: "Qwen3-ASR 不暴露 temperature/beam 参数"
```

7.5 在评估中使用协议

手动脚本

```
# 使用 strict_core (默认)
python scripts/generate_predictions_via_server.py \
    --model-dir src/sure_eval/models/asr_qwen3 \
    --dataset aishell1 \
    --run-dir /tmp/eval_run \
    --protocol strict_core      # 默认, 可省略

# 禁用协议系统
python scripts/generate_predictions_via_server.py \
    ... \
    --protocol none
```

Agent Flow

Agent 在执行 `EXECUTION_SURFACE_UNIT` 时会自动：1. 读取 `config/protocols.yaml` 中的协议定义 2. 读取模型的 `config.yaml` 中的协议映射 3. 解析标准参数到模型参数 4. 通过环境变量 `SURE_EVAL_PROTOCOL_*` 和 `SURE_EVAL_MODEL_*` 注入

模型 Server 可以读取这些环境变量来调整推理行为。

7.6 向后兼容

协议系统是可选的加分项，不影响原有功能：- 旧模型不写 `protocols` 字段 → 正常工作 - 不传 `--protocol` 参数 → 使用默认 `strict_core`，静默回退 - 协议解析失败 → 不影响评估执行

8. 结果解读与报告

8.1 核心指标

WER (Word Error Rate, 词错误率)

WER 是 ASR 评测的核心指标，计算公式：

$$WER = (S + D + I) / N$$

- **S** (Substitutions)：替换错误数

- **D** (Deletions) : 删除错误数
- **I** (Insertions) : 插入错误数
- **N** (Total Words) : 参考答案总词数

中文场景：由于中文没有空格分词，通常使用 **CER** (Character Error Rate , 字错误率)。

RPS (Relative Performance Score)

RPS 是 SURE-EVAL 引入的相对性能分数，表示模型相对于 SOTA 基线的表现：

$$\text{RPS} = (\text{SOTA_score} - \text{Model_score}) / \text{SOTA_score} \times 100$$

- $\text{RPS} > 0$: 优于 SOTA
- $\text{RPS} = 0$: 等于 SOTA
- $\text{RPS} < 0$: 劣于 SOTA

示例：- SOTA WER = 5.0% - 你的模型 WER = 5.5% - $\text{RPS} = (5.0 - 5.5) / 5.0 \times 100 = -10.0$

8.2 评估结果文件

执行 `evaluate_predictions.py` 后，会生成以下文件：

```
eval_runs/<run_id>/
├─ predictions/
│   ├─ aishell1.txt           # 预测结果 (key<TAB>prediction)
│   └─ librispeech_test_clean.txt
│   └─ logs/
│       ├─ aishell1.log       # server stderr
│       └─ aishell1_results.log # 实时结果日志
├─ eval_result.json          # 评估指标结果
├─ assessment_report.json    # Agent 评估报告
└─ main_agent_run_report.json # 最终运行报告
```

8.3 结果示例

```
{
  "dataset": "aishell1-test_ASR",
  "task": "ASR",
  "language": "zh",
  "metric": "cer",
  "score": 4.32,
  "score_unit": "%",
  "num_samples": 7176,
  "sota_baseline": {
    "model": "wenet_conformer_aishell1",
    "score": 4.62
  },
  "rps": 6.49,
  "is_sota": true
}
```

解读： - CER = 4.32%，在 7176 个样本上计算 - SOTA 基线 CER = 4.62% - RPS = 6.49（正值，表示优于 SOTA） - **is_sota: true**：当前模型在该数据集上达到 SOTA

9. 常见问题与故障排除

Q1：模型 server 启动失败

现象：`generate_predictions_via_server.py` 报错 "Server exited before returning a response"

排查步骤：1. 检查模型虚拟环境是否存在：`bash ls src/sure_eval/models/asr_qwen3/.venv/bin/python` 2. 手动测试 server 启动：`bash cd src/sure_eval/models/asr_qwen3 .venv/bin/python server.py # 手动发送 JSON-RPC 请求测试` 3. 检查 `config.yaml` 中的 `server.command` 是否正确 4. 查看 `predictions/logs/<dataset>.log` 中的错误信息

Q2：预测文件为空

现象：评估结果中所有预测为空字符串

排查步骤：1. 检查模型是否正确加载（查看 log 中的 "Model loaded"） 2. 检查音频路径是否正确解析 3. 运行冒烟测试，用单条样本验证：`bash python scripts/generate_predictions_via_server.py \ --model-dir src/sure_eval/models/asr_qwen3 \ --dataset aishell1 \ --run-dir /tmp/test \ --max-samples 1`

Q3 : 下载数据集很慢

解决方案： 1. 使用 ModelScope 国内镜像：`bash export MODELSCOPE_CACHE=./data/cache` 2. 仅下载需要的数据集子集：`bash cd data/datasets/sure_benchmark/SURE_Test_Suites modelscope download --dataset SUREBenchmark/SURE_Test_Suites aishell-1_test.tar.gz`

Q4 : Agent Flow 中冒烟测试失败

现象： SMOKE_TEST_UNIT 报告 "smoke test failed"

解决方案： 1. 检查模型 server 是否能处理单条请求 2. 检查数据集样本格式是否正确 3. 确认 `--max-samples` 没有设置过小导致无法验证 4. 手动运行单条预测验证：`bash cd src/sure_eval/models/asr_qwen3 .venv/bin/python -c "from model import ASRQwen3Model; m = ASRQwen3Model(); print(m.transcribe('path/to/audio.wav').text)"`

Q5 : 评估指标异常 (WER/CER 过高)

现象： ASSESSMENT_UNIT 检测到异常，暂停请求确认

排查步骤： 1. 检查预测文件中的样本是否与参考答案对齐 2. 检查语言设置是否正确（中文数据集用 CER，英文用 WER） 3. 检查文本后处理是否正常（标点、大小写等） 4. 查看 `predictions/logs/<dataset>_results.log` 中的原始预测

Q6 : 如何断点续跑

方案： 使用 `--resume` 参数

```
python scripts/generate_predictions_via_server.py \
  --model-dir src/sure_eval/models/asr_qwen3 \
  --dataset aishell1 \
  --run-dir /tmp/eval_run \
  --resume
```

已完成的样本会自动跳过，继续处理剩余样本。

禁用 resume： 设置环境变量 `NO_RESUME=1`

Q7 : GPU 内存不足

解决方案： 1. 修改 `config.yaml` 中的 `DEVICE` 为 `"cpu"` 2. 或者使用更小的批大小 3. 对于 Qwen3-ASR，当前只支持 `batch_size=1`

Q8 : TOOL_READINESS_AND_ROUTING_UNIT 返回异常状态

现象：主流程 Agent 在工具就绪检查阶段停止，报告 `server_declared_but_unverified` 或 `tool_broken_needs_repair`。

状态含义与处理：

<code>server_declared_but_unverified</code>	<code>config.yaml</code> 中声明了 server，但尚未通过冒烟测试	先手动运行 <code>scripts/generate_predictions_via_server.py --max-samples 1</code> 验证 server 可调用
<code>tool_broken_needs_repair</code>	模型目录或环境损坏，无法完成基本推理	将目标交给 Model Tool Agent 进行修复或重新接入，不要继续执行评估
<code>not_tool_ready</code>	模型尚未接入，缺少 <code>config.yaml</code> 、 <code>server.py</code> 等关键文件	先使用 Model Tool Agent 完成第一阶段 onboarding
<code>server_ready</code>	工具已通过验证	继续执行后续评估步骤

原则：主流程 Agent 不负责修复模型环境。一旦工具未就绪，应当停止评估路由并转交 Model Tool Agent。

10. 附录

附录 A：核心脚本参考

<code>prepare_sure_dataset.py</code>	准备数据集	<code>--dataset</code> , <code>--all</code> , <code>--config</code>
<code>materialize_predictions_template.py</code>	生成预测模板	<code>--dataset</code> , <code>--output-dir</code>
<code>generate_predictions_via_server.py</code>	通过 MCP Server 生成预测	<code>--model-dir</code> , <code>--dataset</code> , <code>--run-dir</code> , <code>--tool-name</code> , <code>--language</code> , <code>--resume</code> , <code>--max-samples</code> , <code>--protocol</code>
<code>validate_prediction_files.py</code>	验证预测文件	<code>--dataset</code> , <code>--pred-dir</code> , <code>--require-nonempty</code>
<code>evaluate_predictions.py</code>	计算指标	<code>--dataset</code> , <code>--pred-dir</code> , <code>--tool-name</code> , <code>--record</code> , <code>--output</code>
<code>refresh_report_snapshot.py</code>	刷新报告快照	<code>--markdown</code> , <code>--json</code>
<code>run_sure_evaluation.py</code>	端到端评估	<code>--gt</code> , <code>--pred</code> , <code>--task</code>

附录 B : 配置文件参考

模型 config.yaml 完整字段

```

name: model_name          # 模型标识名
task: ASR                  # 任务类型
description: "..."       # 描述
version: "1.0.0"

model:
  id: "org/model"          # 模型 ID
  size: "1.7B"
  languages: ["zh", "en"]  # 支持的语言
  license: "Apache 2.0"

server:
  command: ["python", "server.py"] # 启动命令
  working_dir: "."                 # 工作目录
  env:                             # 环境变量
    MODEL_PATH: "..."
    DEVICE: "auto"
  timeout: 300
  startup_timeout_sec: 300

tools:
  - name: "tool_name"
    description: "..."
    input_schema:
      type: object
      properties:
        param1:
          type: string
          description: "..."
      required: ["param1"]

protocols:
  strict_core:
    enabled: true
  param_map:
    precision:
      model_param: dtype
      mapping:
        float16: "torch.float16"
  attestation:
    single_pass_decode: true

resources:
  memory_gb: 8
  gpu: true

```

```
cpu_cores: 4  
storage_gb: 4
```

附录 C : 项目目录结构速查

```

sure-eval/
├── src/sure_eval/
│   ├── agent/                                # 主流程 Agent 编排与状态机实现
│   │   ├── orchestrator.py                 # 编排器
│   │   └── evaluator.py                     # 评估器
│   ├── core/                                # 核心工具
│   │   ├── config.py                       # 配置管理
│   │   └── logging.py                      # 日志
│   ├── datasets/                            # 数据集管理
│   │   └── dataset_manager.py
│   ├── evaluation/                          # 评估指标
│   │   ├── sure_evaluator.py
│   │   └── rps.py                         # RPS 计算
│   ├── inference/                          # 推理层
│   │   ├── runner.py                      # 预测执行
│   │   └── adapters.py                   # 适配器
│   ├── models/                             # 模型目录
│   │   ├── registry.py                   # 模型注册表
│   │   └── asr_qwen3/                   # Qwen3-ASR 示例
│   │       ├── config.yaml
│   │       ├── model.py
│   │       └── server.py
│   └── ...
├── protocols/                              # 推理协议
│   ├── schema.py
│   └── resolver.py
├── reports/                                # 报告生成
├── scripts/                               # 确定性脚本
├── config/                                # 全局配置
│   └── protocols.yaml                    # 协议定义
├── data/datasets/                         # 数据集存储
├── fixtures/tasks/                       # 按任务划分的共享 smoke fixtures
├── docs/agents/                          # Agent workflow 文档、契约与模板（详见下文）
└── eval_runs/                            # 评估运行记录

# docs/agents/ 详细结构
docs/agents/
├── README.md                            # Agent 文档总览
├── main_flow_agent/                     # 主流程评估 Agent
│   ├── AGENTS.md                       # 主流程路由规范
│   ├── README.md                       # 使用指南
│   ├── contracts/                      # 各 UNIT 契约
│   └── templates/                      # 结构化输出与 shell 模板
└── model_tool_agent/                   # 模型接入 Agent
    ├── AGENTS.md                       # 模型接入规范
    ├── README.md                       # 使用指南
    └── contracts/                      # 验证与产物契约

```

```
|— playbooks/           # 后端、预检、失败分类指南
|— policies/           # 决策与安全策略
|— specs/              # 模型 spec 与 wrapper 契约
|— task_playbooks/     # ASR / SD / SER / TTS 等任务手册
|— templates/         # model.spec.yaml、验证脚本等模板
```

附录 D：快速开始清单

如果你是第一次使用 SURE-EVAL，按以下清单操作：

- [] 1. 克隆仓库并安装主环境 (`uv pip install -e .`)
- [] 2. 下载 SURE Benchmark 数据集 (`python scripts/download_sure_data.py`)
- [] 3. 确认模型已接入 (`python -m sure_eval.models.registry`)
- [] 4. 如模型未接入，先走 Model Tool Agent 流程
- [] 5. 选择评估方式：手动脚本 or Agent Flow
- [] 6. 执行评估
- [] 7. 查看结果 (`eval_result.json`)

本手册基于 SURE-EVAL 当前版本编写，如有更新请参考项目文档。